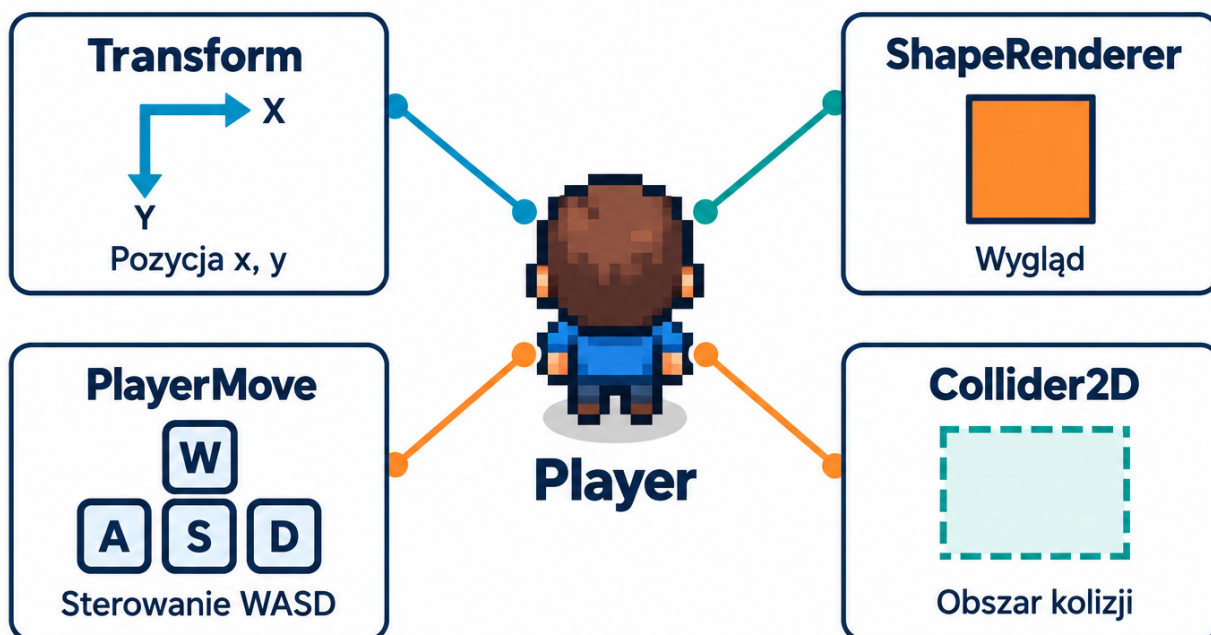


Pierwsza gra, krok po kroku

Trzy lekcje, które przygotują Cię do stworzenia postaci poruszającej się po planszy. Pracujesz w playgroundzie GameDev JS w Web Learning Lab.

Obiekt i komponenty



Jeden obiekt, różne zadania

Po tym kursie potrafisz

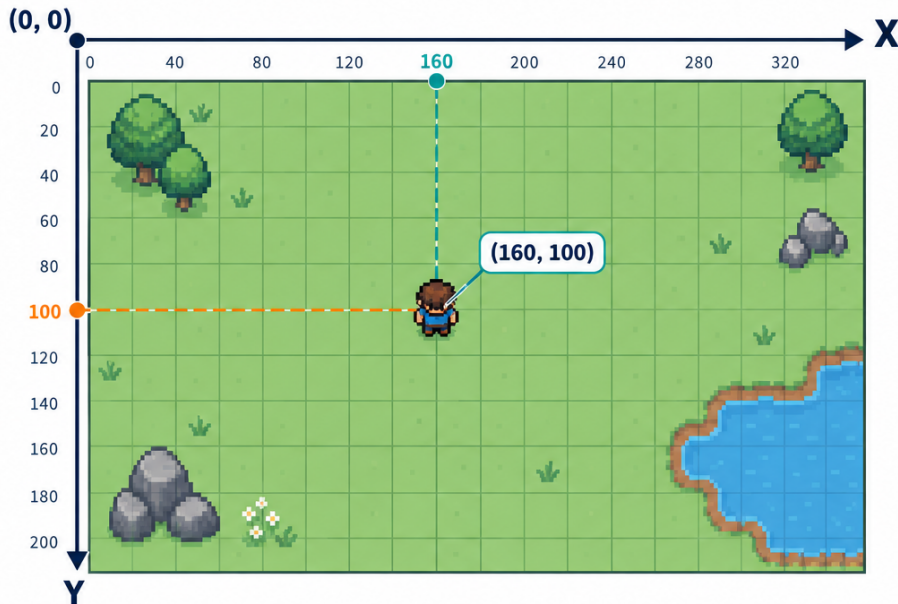
1. Odczytać pozycję (x, y) i przewidzieć kierunek ruchu.
2. Rozdzielić tworzenie obiektów od aktualizacji ich stanu.
3. Złożyć gracza z komponentów i napisać sterowanie WASD.

Organizacja: trzy lekcje po około 25–35 minut. Najpierw przewiduj wynik, potem uruchamiaj kod i porównuj go ze swoją odpowiedzią. Wymagana jest podstawowa znajomość zmiennych, klas i instrukcji if.

Gdzie znajduje się gracz?

Układ współrzędnych

W grach komputerowych używamy układu współrzędnych, w którym początek (0, 0) znajduje się w lewym górnym rogu, oś X rośnie w prawo, a oś Y rośnie w dół.



Ważne!

Pozycja obiektu w grze (np. gracza) określa położenie jego środka, czyli punktu w centrum postaci.

Przykłady ruchu

Ruch w prawo **+X**



Ruch w dół **+Y**



Ruch w górę **-Y**



Bez przesuniętej kamery: (0, 0) to lewy górny róg planszy. X rośnie w prawo, Y rośnie w dół. W GameLab pozycja obiektu oznacza jego środek — nie lewy górny narożnik obrazka.

```
const player = this.createObject('Player');
player.setPosition(160, 100);
```

Pierwsza liczba to **x**, druga to **y**. Zmiana x nie zmienia y. Przy aktywnej kamerze pozycja w świecie nie musi być identyczna z pozycją na ekranie.

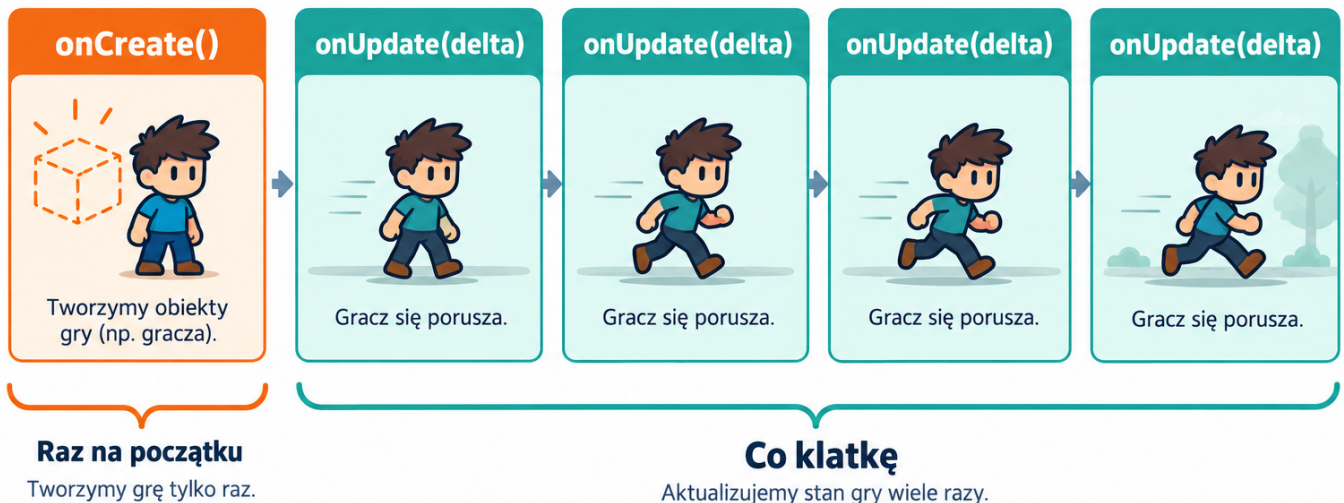
Przewidź, zanim uruchomisz

- Gracz jest w (160, 100). Gdzie będzie po ruchu o 40 px w prawo?
- Z tego samego punktu przesuń go o 30 px w górę.
- Na planszy 640 × 360 px znajdź środek.
- Kwadrat 40 × 40 px ma środek w (100, 80). Gdzie jest jego lewy górny róg?

Twórz raz. Aktualizuj co klatkę.

Cykl życia gry

Tworzymy raz, aktualizujemy wiele razy.



~60 klatek/s

Tempo zależy od urządzenia.

onCreate() silnik wywołuje raz przy uruchomieniu danej sceny. Każdy nowy komponent też ma własne **onCreate()**, wywołane raz podczas jego inicjalizacji. Restart tworzący nową scenę uruchamia tę metodę ponownie.

onUpdate(delta) silnik wywołuje co klatkę. Przy 60 FPS to około 60 wywołań na sekundę, ale nie jest to stała gwarancja. Częstotliwość zależy m.in. od przeglądarki, ekranu i wydajności.

Co należy do której metody?

onCreate: stwórz gracza, ustaw startową pozycję, dodaj renderer i komponent ruchu.

onUpdate: odczytaj klawisze, oblicz ruch, zmień stan gry.

Uwaga: tworzenie gracza w każdej aktualizacji produkuje nowe obiekty zamiast poruszać jednym.

Ruch zależy od czasu, nie od FPS

delta to czas kroku aktualizacji w sekundach. Przy 60 FPS wynosi zwykle około 1/60 s. Prędkość 120 px/s oznacza przesunięcie o około 2 px na taką klatkę.

```
class AutoMove extends GameLab.Component {  
  onUpdate(delta) {  
    this.transform.x += 120 * delta;  
  }  
}
```

Wzór: **przesunięcie = prędkość × czas**. Przy 30 FPS delta wynosi około 1/30 s: przesunięcie to 4 px na klatkę. Po sekundzie w obu przypadkach gracz przesunie się około 120 px, jeśli nic go nie zatrzyma.

Najczęstszy błąd

```
this.transform.x += 2; // 2 px na klatkę, nie na sekundę
```

Ten zapis daje około 120 px/s przy 60 FPS, ale tylko 60 px/s przy 30 FPS. Unikaj go, gdy chcesz zachować tę samą prędkość na różnych komputerach.

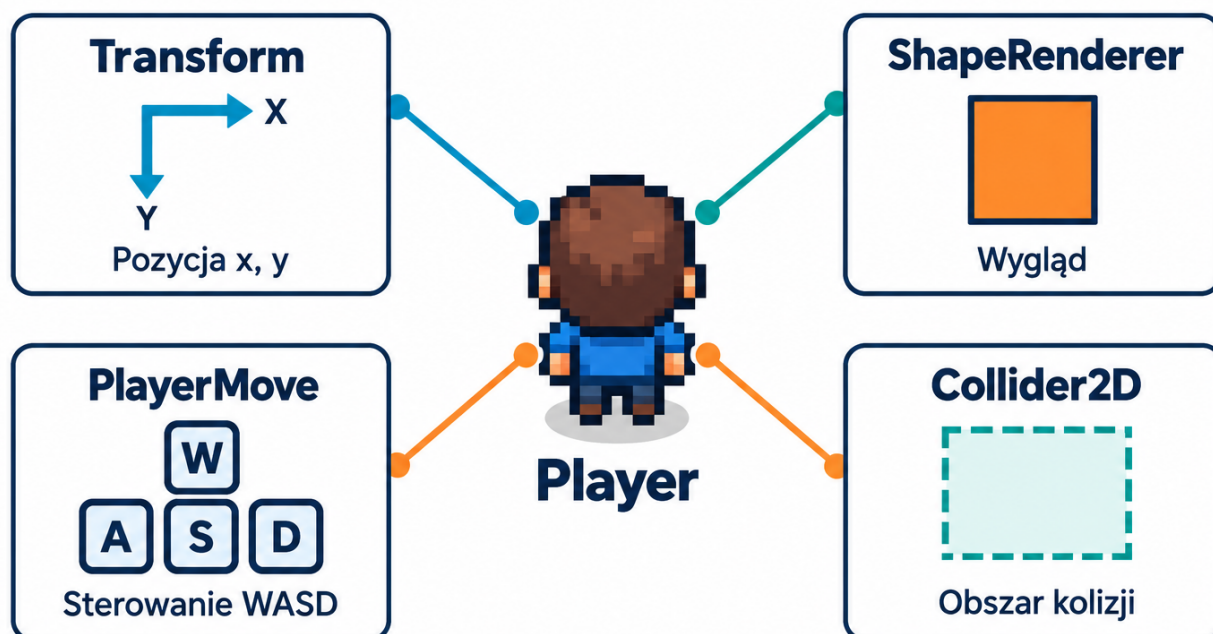
Ćwiczenia

1. Oblicz przesunięcie dla 90 px/s i delta = 0,02 s.
2. Zmień AutoMove tak, by obiekt poruszał się w dół.
3. Zmień kierunek na lewo i ustaw 80 px/s.
4. Wyjaśnij, dlaczego nie dodajemy nowego gracza w onUpdate.

Ważne w GameLab: CharacterController2D.move(x, y, speed) przyjmuje prędkość w px/s. Silnik sam uwzględnia delta. Nie mnożysz speed przez delta drugi raz.

Z czego składa się gracz?

Obiekt i komponenty



Jeden obiekt, różne zadania

GameObject to obiekt, np. **Player**. **Transform** przechowuje jego pozycję i jest dostępny jako `player.transform`. Nie musisz dodawać go ręcznie. Pozostałe zachowania dołączasz przez `addComponent()`.

ShapeRenderer: rysuje kształt.

PlayerMove: Twoja klasa, która czyta klawisze.

CharacterController2D: wykonuje ruch i obsługuje blokowanie przez collidery.

Collider2D: określa obszar kolizji. Samo dodanie go nie tworzy sterowania.

Komponenty odpowiadają na pytanie „co ten obiekt potrafi?”. Ten sam **PlayerMove** można dołączyć do innego obiektu. Usunięcie renderera nie usuwa obiektu — może nadal mieć pozycję i zachowanie, choć będzie niewidoczny.

Zbuduj scenę i gracza

W pustym projekcie GameDev JS umieść poniższy kod w pliku uruchamiającym grę. Dodaj klasę `PlayerMove` z następnej strony **przed** wywołaniem `GameLab.run()`.

```
class MyGame extends GameLab.Game {
  onCreate() {
    const player = this.createObject('Player');
    player.setPosition(160, 100);

    const renderer = new GameLab.ShapeRenderer({
      width: 32,
      height: 32,
      color: '#008f95'
    });
    player.addComponent(renderer);
    player.addComponent(
      new GameLab.CharacterController2D()
    );
    player.addComponent(new PlayerMove());
  }
}

// Tutaj dodaj klasę PlayerMove z kolejnej strony.
GameLab.run(MyGame);
```

Sprawdź wynik

Po uruchomieniu zobaczysz jeden kwadrat. Jego środek ma pozycję (160, 100). Renderer zapewnia wygląd, ale dopiero `PlayerMove` wyznaczy kierunek ruchu. Kliknij planszę, aby nadać jej fokus, a potem używaj WASD.

Kolejność jest celowa: najpierw `createObject()`, potem `setPosition()`, potem komponenty. Każdy krok ma osobną instrukcję, więc łatwiej znaleźć błąd.

Dodaj sterowanie WASD

```
class PlayerMove extends GameLab.Component {
  onUpdate(delta) {
    const controller = this.gameObject.getComponent(
      GameLab.CharacterController2D
    );
    const input = this.game.input;
    let x = 0;
    let y = 0;

    if (input.isKeyDown('a')) x -= 1;
    if (input.isKeyDown('d')) x += 1;
    if (input.isKeyDown('w')) y -= 1;
    if (input.isKeyDown('s')) y += 1;

    controller.move(x, y, 120);
  }
}
```

W i S zmieniają kierunek na osi Y. A i D zmieniają kierunek na osi X. Gdy nie wciskasz klawiszy, kierunek wynosi (0, 0). Kontroler normalizuje ruch po skosie, aby nie był szybszy.

Parametr delta zostaje tu dla pokazania cyklu życia, ale nie używamy go bezpośrednio: kontroler uwzględnia czas wewnątrz silnika. **PlayerMove jest naszą klasą**, nie gotową metodą GameLab.

Samodzielne zmiany

1. Ustaw start w środku planszy 640 × 360.
2. Zwiększ prędkość do 180 px/s.
3. Zmień kolor i rozmiar gracza.
4. Usuń na chwilę PlayerMove i opisz, co nadal działa.
5. Nie dodawaj AutoMove równocześnie: to inne ćwiczenie ruchu.

Sprawdź, czy rozumiesz

Mini-projekt: dwa obiekty

Stwórz Player i Marker. Player ma kwadrat 32×32 px, startuje w (80, 80) i porusza się przez WASD z prędkością 120 px/s. Marker to pomarańczowy kwadrat 24×24 px w (320, 180), bez sterowania. Nie musisz jeszcze sprawdzać dotarcia do celu.

Kryteria: dwa obiekty powstają w onCreate; oba mają renderer; tylko Player ma PlayerMove i kontroler; po puszczeniu klawiszy Player się zatrzymuje; ruch w górę zmniejsza y. Projekt nie tworzy dodatkowych graczy w każdej klatce.

Powiedz własnymi słowami

Dlaczego punkt (100, 50) jest wyżej niż (100, 150)?

Co stanie się po ponownym uruchomieniu sceny?

Czym różni się obiekt od komponentu?

Dlaczego renderer nie wystarcza do sterowania?

Czy onUpdate zawsze wykona się dokładnie 60 razy na sekundę?

Odpowiedzi do ćwiczeń

Współrzędne: A: (200, 100). B: (160, 70). C: (320, 180). D: (80, 60).

Czas: $90 \times 0,02 = 1,8$ px. Ruch w dół: zwiększaj y. Ruch w lewo: odejmuj $80 \times \text{delta}$ od x.

Cykl: tworzenie w każdej klatce mnoży obiekty. Restart nowej sceny ponownie wywołuje onCreate. 60 FPS to przykład, nie gwarancja.